

Lab 4 - Differential Privacy 1

In the following two lab sessions, we're going to build a general understanding of Differential Privacy in the context of a database query. We are then going to learn how to know whether a database query over such a small database is differentially private or not - and more importantly - what techniques are at our disposal to ensure various levels of privacy

Objectives

1. Create a simple database and generate parallel databases
2. Create query functions and calculate sensitivity
3. Attempt to complete several differential attacks (Review Lab 3)
4. Implement **local differential privacy**

1 Preparation

1.1 Step 1: create a simple database

First, create our database. The database is going to be a VERY simple database with only one boolean column. Each row corresponds to a person. Each value corresponds to whether or not that person has a certain private attribute (such as whether they have a certain disease, or whether they are above/below a certain age).

We're going to do this by initializing a random list of 1s and 0s (which are the entries in our database). Note - the number of entries directly corresponds to the number of people in our database.

```
[1]: import numpy as np

# the number of entries in our database
num_entries = 1000

db = np.random.random_sample(num_entries) > 0.5
# the shape of the numpy array
print(db.shape)
db
```

```
(1000,)
```

```
[1]: array([ True, False, False, False, False,  True, False, False, False,
         True,  True, False, False, False, False, False, False,  True,
        False, False,  True,  True,  True,  True,  True,  True,  True,
        True, False,  True, False, False,  True,  True,  True,  True,
        True, False, False, False, False,  True, False, False, False,
        True,  True, False, False, False, False, False,  True,  True,
        True,  True,  True,  True,  True, False, False, False,  True,
        True, False, False,  True,  True,  True,  True, False,  True,
        False, False, False, False, False,  True, False,  True,  True,
        True,  True,  True, False,  True, False, False,  True, False,
        False,  True, False, False, False, False, False, False,  True,
        True,  True, False,  True, False, False, False,  True,  True,
        False,  True,  True, False, False,  True, False,  True, False,
        False,  True,  True, False, False, False, False,  True,  True,
        True, False, False, False, False,  True, False, False,  True,
        False,  True, False, False, False, False, False, False,  True,
        True, False, False, False, False,  True,  True,  True, False,
        False,  True,  True, False, False,  True, False,  True,  True,
        False,  True,  True, False, False, False,  True,  True,  True,
        True, False, False,  True, False, False, False, False, False,
        False,  True, False,  True,  True, False,  True, False,  True,
        True, False,  True,  True,  True,  True, False,  True,  True,
        True,  True,  True,  True, False,  True, False, False, False,
        False,  True, False, False,  True,  True, False, False,  True,
        False,  True, False, False,  True, False, False,  True,  True,
        False,  True, False, False,  True, False,  True,  True, False,
        False,  True,  True, False,  True,  True,  True,  True,  True,
        True, False, False,  True,  True, False, False,  True,  True,
        True,  True, False,  True,  True, False, False,  True, False,
        False,  True,  True, False,  True, False, False, False,  True,
```



```
False, False, False, False, False, True, False, True, False,
True, False, True, True, False, False, False, False, False,
True, False, True, True, False, True, True, True, True,
True, True, False, False, True, True, True, True, True,
True, False, True, False, True, True, False, False, True,
False, True, False, False, False, False, False, False, True,
True, True, True, True, True, False, True, False, True,
False, True, False, False, False, True, False, False, False,
True, True, True, False, False, False, False, False, True,
True, True, True, True, True, False, False, False, True,
False, True, True, False, False, False, True, True, False,
False, True, True, False, True, True, True, False, False,
True, True, True, True, False, False, True, False, False,
True, False, False, False, True, True, True, True, False,
True, True, True, False, False, True, True, True, False,
False, True, True, True, True, True, False, False, False,
True, False, True, False, False, True, False, True, False,
False])
```

1.2 Step 2: generate parrallel databases

Key to the definition of differential privacy is the ability to ask the question “When querying a database, if I removed someone from the database, would the output of the query be any different?”. Thus, in order to check this, we must construct what we term “parallel databases” which are simply databases with *one entry* removed.

```
[2]: def get_parrallel_db(db, remove_index):
      pdb = list(db)
      # remove the entry by index
      del pdb[remove_index]
      return np.array(pdb)

[3]: get_parrallel_db(db, 2) # generate a parralleb database by removing the 2nd
      ↪entry in the intial database
      print('Size of intial db:', len(db))
      print('Size of parrallel db:', len(get_parrallel_db(db, 2)))
```

Size of intial db: 1000

Size of parrallel db: 999

Then we try build a function that **generates N parrallel databases at once** where N is the length of the initial database.

```
[4]: def get_parrallel_dbs(db):
      parrallel_dbs = list()
      for i in range(len(db)):
          # each parrallel database removes the ith entry from the intial database
```

```

        pdb = get_parallel_db(db,i)
        parrallel_dbs.append(pdb)
    return parrallel_dbs

```

Now to make it more convenient for the later tasks, we can build a function that can **jointly build an initial database and generates all the possible parrallel database at once.**

```
def create_db_and_parrallels(num_entries):
    db = np.random.random_sample(num_entries) > 0.5
    pddb = get_parallel_dbs(db)
    return db, pddb
```

For example, we can build a database with 20 entries and check the function result.

```
db, pdirs = create_db_and_parrallels(20)
print('initial database:\n', db)
print('parrallel databases:\n', pdirs)
```

```
initial database:
```

```
[False True False False False False False False False False True
 True True False True True True True False]
```

parallel databases:

[illegible]

```

False]), array([False,  True, False, False, False, False, False, False,
False,
False,  True,  True,  True, False,  True,  True,  True,  True,
False]), array([False,  True, False, False, False, False, False, False,
False,
False,  True,  True,  True, False,  True,  True,  True,  True,
False]), array([False,  True, False, False, False, False, False, False,
False,
False, False,  True,  True, False,  True,  True,  True,  True,
False]), array([False,  True, False, False, False, False, False, False,
False,
False, False,  True,  True, False,  True,  True,  True,  True,
False]), array([False,  True, False, False, False, False, False, False,
False,
False, False,  True,  True,  True,  True,  True,  True,  True,
False]), array([False,  True, False, False, False, False, False, False,
False,
False, False,  True,  True,  True, False,  True,  True,  True,
False]), array([False,  True, False, False, False, False, False, False,
False,
False, False,  True,  True,  True, False,  True,  True,  True,
False]), array([False,  True, False, False, False, False, False, False,
False,
False, False,  True,  True,  True, False,  True,  True,  True,
True]])]

```

1.3 Step 3: calculate the sensitivity in terms of query functions

Intuitively, we want to be able to query our database and evaluate whether or not the result of the query is leaking “private” information. As mentioned previously, this is about evaluating whether the output of a query changes when we remove someone from the database. Specifically, we want to evaluate the maximum amount the query changes when someone is removed (maximum over all possible people who could be removed). So, in order to evaluate how much privacy is leaked, we’re going to iterate over each person in the database and measure the difference in the output of the query relative to when we query the entire database.

So let us create an initial database with 1000 entries and 999 parrallel databases with 999 entries, and then make our first “database query” a simple *sum*.

```
[7]: def query_sum(db):  
      return db.sum()
```

```
[8]: def sensitivity_sum(n_entries):  
      # generate the initial database and all the possible parrallel databases  
      db, pddb = create_db_and_parrallels(n_entries)  
  
      # sum value of the intial database  
      full_db_result = query_sum(db)  
  
      maximum_distance = 0  
      for pdb in pddb:  
          # sum value of each parrallel database  
          pdb_result = query_sum(pdb)  
  
          # the difference between the sum values of the initial and each  
          ↪parrallel database  
          db_distance = np.abs(pdb_result-full_db_result)  
          # find out and return the maximum difference from all those differences  
          if(db_distance > maximum_distance):  
              maximum_distance = db_distance  
      return maximum_distance
```

```
[9]: sensitivity_sum(1000)
```

```
[9]: 1
```

The maximum difference between query results from the intial database and each parrallel database is 1, therefore the sensitivity is 1. This value is called “sensitivity”, and it corresponds to the function we chose for the query. Namely, the “sum” query will always have a sensitivity of exactly 1. However, we can also calculate sensitivity for other functions as well.

Let’s try to calculate sensitivity for the *mean* function.

```
[10]: # calculate the mean value of the entries in a dataset  
def query_mean(db):  
    return db.mean()
```

```
[11]: def sensitivity_mean(n_entries):  
      # generate the initial database and all the possible parrallel databases  
      db, pddb = create_db_and_parrallels(n_entries)  
  
      # mean value of the intial database  
      full_db_result = query_mean(db)  
  
      maximum_distance = 0  
      for pdb in pddb:  
          # mean value of each parrallel database
```

```

    pdb_result = query_mean(pdb)

    # the difference between the mean values of the initial and each
    ↪parallel database
    db_distance = np.abs(pdb_result-full_db_result)

    # find out and return the maximum difference from all those differences
    if(db_distance > maximum_distance):
        maximum_distance = db_distance
    return maximum_distance

```

```
[12]: sensitivity_mean(1000)
```

```
[12]: 0.0005115115115115199
```

Now, I want you to calculate the sensitivity for the “threshold” function.

First compute the sum over the database (i.e. `sum(db)`) and return whether that sum is greater than a certain threshold. Then, I want you to create databases of size 10 and threshold of 5 and calculate the sensitivity of the function. Finally, re-initialize the database 10 times and calculate the sensitivity each time.

```
[13]: def query_threshold(db, threshold=5):
    if db.sum()>threshold:
        return 1
    else:
        return 0

```

```
[14]: def sensitivity_threshold(n_entries):
    # generate the initial database and all the possible parallel databases
    db, pddb = create_db_and_parrallels(n_entries)

    # boolean value if the sum value of the initial database is greater than
    ↪the threshold
    full_db_result = query_threshold(db)

    maximum_distance = 0
    for pdb in pddb:
        # boolean value if the sum value of each parallel database is greater
        ↪than the threshold
        pdb_result = query_threshold(pdb)

        # the difference between the boolean values of the initial and each
        ↪parallel database
        db_distance = np.abs(pdb_result-full_db_result)

        # find out and return the maximum difference from all those differences

```



```

        if(db_distance > maximum_distance):
            maximum_distance = db_distance
    return maximum_distance

```

```

[15]: for i in range(10):
        print(sensitivity_threshold(10))

```

```

0
0
1
1
1
0
0
0
0
0

```

Now you can see that the sensitivity is variable rather than constant. So no matter if the sensitivity is constant or variable, please remember that the sensitivity is just based on the query function that you have.

1.4 Step 4: define a differential attack

Sadly none of the functions we've looked at so far are differentially private (despite them having varying levels of sensitivity). The most basic type of attack can be done as follows.

Let's say we wanted to figure out a specific person's value in the database. All we would have to do is query for the sum of the entire database and then the sum of the entire database without that person!

Let us create a database with 100 entries.

```

[16]: db,_ = create_db_and_parallel(100)

```

For example, let us perform a differential attack on **Row 10**. So let us create a parallel database where just the **Row 10** is missing.

```

[17]: pdb = get_parallel_db(db, remove_index=10)

```

Let us check what is the value of **Row 10** in the initial database.

```

[18]: db[10]

```

```

[18]: False

```

```

[19]: # differential attack using sum query
        sum(db) - sum(pdb)

```

[19]: 0

From the sum query attack, we can know that if Row 10 is True (1), then the sum value is 1, otherwise is 0.

```
[20]: # differential attack using mean query
      sum(db)/len(db) -sum(pdb)/len(pdb)
```

[20]: -0.004040404040404011

From the mean query attack, we can know that if Row 10 is True (1), then the mean value is none-zero, otherwise is 0.

```
[21]: # differential attack using threshold
      threshold = sum(db) - 1
      sum(pdb) > threshold
```

[21]: True

So from the threshold query attack, we can know that if Row 10 is True (1), then the sum(pdb) should not be greater than the threshold, otherwise it is greater than the threshold.

2 Quiz

2.0.1 1. Create a database with 2000 entries and generate all the parrallel databases.

2.0.2 2. Create two queries - one is to calculate the sum of values on all the even rows, and another is to calculate the mean value of all the odd rows, and calculate the corresponding sensitivities.

2.0.3 3. Perform the corresponding differential attacks on that database.

2.1 Step 5: perform local differential privacy (major)

2.1.1 Randomied Response (Local Differential Privacy)

Let's say we have a group of people we wish to survey about a very taboo behaviour which they would lie about. So, how do we do this? One technique is to add randomness to each person's response by giving each person the following instructions:

- i. Flip a coin 2 times.
- ii. If the first coin flip is heads, answer honestly
- iii. If the first coin flip is tails, answer according to the second coin flip (heads for yes, tails for no)!

Thus, each person is now protected with "plausible deniability". If they answer "Yes" to the question "have you committed X crime?", then it might becasue they actually did, or it might be becasue

they are answering according to a random coin flip. Each person has a high degree of protection. Furthermore, we can recover the underlying statistics with some accuracy, as the “true statistics” are simply averaged with a 50% probability. Thus, if we collect a bunch of samples and it turns out that 60% of people answer yes, then we know that the TRUE distribution is actually centered around 70%, because 70% averaged with 50% (a coin flip) is 60% which is the result we obtained.

However, it should be noted that, especially when we only have a few samples, this comes at the cost of accuracy. This tradeoff exists across all of Differential Privacy. The greater the privacy protection (plausible deniability) the less accurate the results.

Let’s implement this local DP for our database before!

```
[22]: db,pdbs = create_db_and_parrallels(100)
```

```
[23]: # this is the true value of people's response
db
```

```
[23]: array([ True, False, False,  True, False,  True, False, False,  True,
         True,  True, False,  True,  True,  True, False, False, False,
        False,  True,  True,  True, False,  True, False, False,  True,
         True,  True,  True, False,  True,  True, False,  True, False,
         True, False,  True, False,  True,  True, False,  True,  True,
        False,  True, False,  True, False, False, False,  True, False,
         True,  True, False, False, False,  True, False, False,  True,
         True,  True,  True, False,  True,  True, False,  True, False,
         True, False,  True,  True,  True,  True, False, False,  True,
         True,  True,  True,  True, False, False,  True,  True, False,
        False, False, False, False,  True, False,  True,  True, False,
        False])
```

```
[24]: def query(db):
    true_result = db.mean()
    # flip the coin for each entry (ie. each person in this case)
    first_coin_flip = (np.random.random_sample(len(db)) > 0.5)
    second_coin_flip = (np.random.random_sample(len(db)) > 0.5)

    # give the answer honestly if the 1st flip is heads,
    # otherwise give the answer according to the 2nd flip
    augmented_database = db*first_coin_flip + (1 - first_coin_flip) *_
↪second_coin_flip

    # try to reverse to the true result
    db_result = (augmented_database).mean() * 2 - 0.5

    return db_result,true_result
```

```
[25]: # try different volumes of dataset below
db, pdbs = create_db_and_parrallels(10)
```

```
private_result,true_result = query(db)
print('With noise:',str(private_result))
print('Without noise:',str(true_result))
```

With noise: 1.1
Without noise: 0.5

You can try different volumes of dataset and then you can see that the larger dataset is, the smaller difference between the query results. It means that the distribution with local differential noise is still centred based on the distribution of the true dataset. Moreover, local differential privacy is a data-hungry mechanism which is more suitable for the dataset with the larger volume.

Now we just use a term called “noise” to replace the first coin flip, which is the probability of giving the true response. Then, we can re-write the query as below:

```
[26]: def query(db, noise):
    true_result = db.mean()
    # flip the coin for each entry (ie. each person in this case)
    first_coin_flip = (np.random.random_sample(len(db)) > noise)
    second_coin_flip = (np.random.random_sample(len(db)) > 0.5)

    # give the answer honestly if random number is greater than noise,
    # otherwise give the answer according to the 2nd flip
    augmented_database = db * first_coin_flip + (1 - first_coin_flip) *
    ↪second_coin_flip

    # try to reverse to the true result
    db_result = ((augmented_database).mean() / noise - 0.5) * noise / (1 -
    ↪noise)

    return db_result,true_result
```

```
[27]: # try different noises below, and implement the following code several times
    ↪per noise
db, pddb = create_db_and_parallel(1000)
private_result,true_result = query(db, noise = 0.1)
print('With noise:',str(private_result))
print('Without noise:',str(true_result))
```

With noise: 0.49666666666666665
Without noise: 0.499

As you can see, the larger noise a dataset has, the greater difference between the query results.